

Desenvolvimento Web III
Professor Jefferson Chaves
jefferson.chaves@ifpr.edu.br

Aprofundando em Aplicações Web

O objetivo desta atividade é a realização de um laboratório para estudo de construção de aplicações web.

A internet se consolidou como principal meio de consumo e troca de informações atualmente. Apoiada sobre uma pilha de protocolos, dentre os quais destaco aqui o TCP e IP, permitiu que dados fossem enviados de um ponto a outro ao redor do mundo.

*Usando a internet como meio para troca de dados, diversas aplicações foram desenvolvidas tais como Email (usando os protocolos SMTP, POP3, IMAP), Transferência de arquivos (usando o protocolo FTP), Compartilhamento de arquivos (protocolo [BitTorrent](#)) e a Web, usando o protocolo HTTP. **Vamos focar na Web.***

*A World Wide Web ou somente web, ganhou popularidade devido a simplicidade. A transferência de dados baseada na arquitetura cliente e servidor embora simples, era robusta o suficiente para as aplicações da época até as aplicações atuais. Seu protocolo, o HTTP, define como essas trocas serão realizadas. A solicitação é realizada indicando-se um **verbo HTTP**, como por exemplo GET, POST, DELETE e etc. A resposta é informada juntamente com um código de estado: [404](#), [500](#), [200](#), [301](#) e etc. Os dados são enviados em “envelopes” HTTP, contendo cabeçalhos e o corpo da mensagem.*

Criar uma aplicação web, e compreender o que está acontecendo nos bastidores envolve conhecer minimamente o protocolo HTTP. Quanto mais se conhece do ecossistemas, mais é possível se extrair de aplicações web, em termos de arquitetura, desempenho, segurança e etc.

Exercício 1

Crie um projeto Java Web. Se não estiver seguro de como fazer isso, pode seguir o guia para criação de projeto disponível no material publicado no tópico “Introdução a aplicações web usando JAVA”, crie um projeto Java Web com Spring;

Exercício 2

Projetos Web Java modernos são desenvolvidos utilizando frameworks que simplificam a configuração e a estrutura da aplicação. O Spring Boot é um dos principais frameworks utilizados atualmente, pois permite criar aplicações web de forma rápida, com configuração automática e baseada em convenções.

Nesse modelo, a estrutura do projeto segue padrões bem definidos, e grande parte das configurações é feita por meio de anotações em classes Java. As requisições HTTP são tratadas por classes chamadas Controllers, responsáveis por receber as requisições e retornar respostas ao cliente.

Questão a: *defina a página inicial da aplicação, criando um Controller que responda à rota principal (/):*

`@Controller`

```
public class HomeController {  
  
    @GetMapping("/")  
    public String home() {  
        return "home";  
    }  
}
```

Nesse caso, o método retorna uma view chamada **home**, que pode ser um arquivo **home.html** localizado na pasta **src/main/resources/templates**.

Execute a aplicação e acesse o endereço principal no navegador para verificar o funcionamento.

Questão b) O tratamento de erros também pode ser personalizado. Para o erro 404 (página não encontrada), basta criar um arquivo chamado **404.html** dentro do diretório:

src/main/resources/templates/error

Suas vez! Nesse arquivo, adicione uma mensagem informando que a página não foi encontrada. Ao acessar uma URL inválida, essa página será exibida automaticamente.

Em seu navegador, acesse **localhost:8080/pagina_inexistente** e veja o que acontece.

Questão c) As requisições da aplicação são processadas por Controllers, que funcionam como **pontos de entrada** para as funcionalidades do sistema. Esses componentes (nesse caso, os controllers) são gerenciados pelo container da aplicação, que também controla seu ciclo de vida.

Para observar esse ciclo de vida, implemente o `LifeCycleController` utilizando as anotações especificadas, conforme o código abaixo:

`@Controller`

```
public class LifeCycleController {

    @PostConstruct
    public void init() {
        System.out.println("*** Chamou o método init ***");
    }

    @PreDestroy
    public void destroy() {
        System.out.println("*** Chamou o método destroy ***");
    }

    @GetMapping("/lifecycle")
    @ResponseBody
    public String service() {
        System.out.println("*** Chamou o método service ***");
        return "";
    }
}
```

Ao acessar a rota `/lifecycle`, a aplicação processa a requisição e executa o método correspondente. O retorno no navegador pode ser vazio, mas o console do seu VSCode exibirá a mensagem indicando a execução, já que estamos usando o comando `System.out.println`

Responda:

- i) Quantas vezes apareceu a mensagem “Chamou o método init”?*
- ii) Quantas vezes apareceu a mensagem “Chamou o método service”?*
- iii) Quantas vezes apareceu a mensagem “Chamou o método destroy”?*
- iv) Atualizando seu navegador muitas vezes, apertando F5 por exemplo, existe algum efeito?*

Questão d) *Implemente um Controller responsável por contabilizar o número de acessos, conforme código abaixo:*

```
@Controller
public class ContadorController {

    private int contador = 0;

    @GetMapping("/contador")
    @ResponseBody
    public String contar() {
        contador++;
        return "Número de acessos: " + contador;
    }
}
```

*Acesse a rota **/contador** diversas vezes e observe o valor retornado. Teste também em diferentes navegadores. Reinicie a aplicação e observe o que acontece com o contador.*

Responda:

- i) Atributos de classe podem ser compartilhados entre diferentes requisições dentro da aplicação. Em quais situações isso pode ser útil?*
- ii) Na sua opinião, essa abordagem é adequada para sistemas com múltiplos usuários acessando o sistema simultaneamente?*
- ii) As informações de uma requisição não são compartilhadas. Dizemos que são “Thread Safe”. Explique a importância desse comportamento.*